

配布資料の内容

6.1	いろいろな制御文	6-1
6.2	ゲームを作ろう	6-4
6.3	演習問題	6-8
6.4	今回の実習内容	6-9

6.1 いろいろな制御文

if 文による場合分け (選択) や、while 文による繰り返し (反復) など、実行の流れを制御しているプログラムの構造を制御構造と言います。また、これら 2 つの文を含めて、break 文など、プログラムの実行の流れを制御するための文を一般に制御文と呼びます。C には、if 文や while 文、break 文の他にも、いろいろな制御文が用意されています。

for 文 C には、決った回数の繰り返しを行うために便利な制御構造として for 文と呼ばれる構文が用意されています。for 文は

```
for( 初期設定式 ; 継続条件式 ; 再設定式 ) 文
```

という形の構文です。もちろん、複数の行に跨って書いても構いません。この for 文全体も 1 つの文となります。for 文は、

```
初期設定式 ;
while( 継続条件式 ) {
    文
    再設定式 ;
}
```

という while 文を使った繰り返しと同じように実行されます。たとえば、

```
sum = 0;
i = 1;
while (i <= n) {
    sum = sum + i;
    i = i + 1;
}
```

という while 文を使ったプログラムで、1 から n までの整数の和を変数 sum に格納することができますが、これと同じことをするプログラムを、for を使って

```
sum = 0;
for (i = 1; i <= n; i = i + 1)
    sum = sum + i;
```

のように簡潔に書くことができます。また、

```
k = 0;
while (k < n) {
    i = 0;
```

```

while (i < n) {
    tForward(50);
    tTurn(-360.0/n);
    i = i + 1;
}
tForward(50);
tTurn(360.0/n);
k = k + 1;
}

```

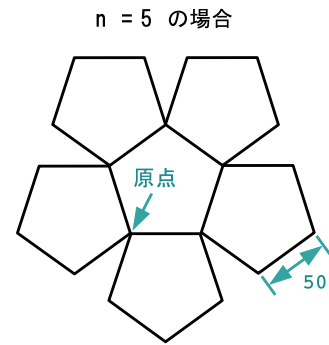
というプログラムは、正 n 角形を (正 n 角形の回りに) n 個描くもの (k と k は整数値の変数) ですが、これと等価なプログラムを、for 文を使って次のように書くこともできます。

```

for (k = 0; k < n; k++) {
    for (i = 0; i < n; i++) {
        tForward(50);
        tTurn(-360.0/n);
    }
    tForward(50);
    tTurn(360.0/n);
}

```

このプログラムの2つの for 文の再設定式で使用されている $k++$ や $i++$ は、それぞれ $k = k + 1$ や $i = i + 1$ と同じ働きをする式です。詳しくは第8回で説明します。



メモ

for 文の初期設定式、継続条件式、再設定式は必要がなければ省略しても構いません (ただし、それでもこれら3つを区切っている ; は必要です)。継続条件式を省略すると、そこに常に成り立つような条件が書かれているのと等価になります。つまり、

for (;;) 文

は

while (1) 文

と等価になります。

前回紹介した break 文は、for 文や、次節で紹介する do 文による繰り返しを途中終了するためにも使うことができます。

メモ

do 文 do 文は、

```
do 文 while( 継続条件式 );
```

という形の構文で、やはり繰り返しの処理を行うために用いられます。while というキーワード (予約語) が含まれていますが、これも do 文の構文の一部であって、決して while 文の始まりではありません。

do 文は、

文

```
while( 継続条件式 ) 文
```

という2つの文からなるプログラムと同じように実行されます。do 文では、繰り返される「文」の部分が、無条件で必ず1回は実行されることに注意してください。

メモ

次の broute.c は、tScanf を使ってキーボードから x 座標と y 座標の値を読み込み、カメを指定された点に次々と移動させていくプログラムですが、原点からの距離が200を超える点に移動すると、この繰り返しの終了し、(ペンを上げて)原点に戻るようになっています。

broute.c

```
#include <turtle.h>
```

```
int main()
```

```
{
```

```
    double x, y;
```

```
    do {
```

```
        tPrintf("座標? ");
```

```
        if (tScanf("%lf%lf", &x, &y) != 2)
```

```
            break;
```

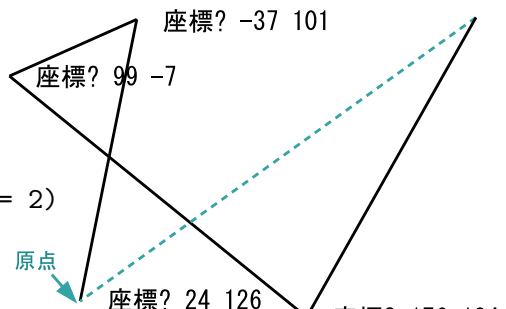
```
        tMoveTo(x, y);
```

```
    } while (x*x + y*y <= 40000);
```

```
    tPenUp();
```

```
    tMoveTo(0, 0);
```

```
}
```



この do 文の部分は

```
x = y = 0;
```

```
while (x*x + y*y <= 40000) {
```

```
    tPrintf("座標? ");
```

```
    if (tScanf("%lf%lf", &x, &y) != 2)
```

```
        break;
```

```
    tMoveTo(x, y);
```

```
}
```

と書き換えても同じように動作しますが、`x = y = 0;` という文による変数の初期化や、`while` 文の条件式の最初のチェックは (本来は必要の無い) 無駄な作業になってしまいますので、`do` 文を使った方がより自然なプログラムになります。

C 言語には、この他にも次節で紹介する `continue` 文や、`switch` 文、`goto` 文といった構文があり、プログラムの実行の流れを制御することができます。詳細は参考書等を参照してください。

メモ

6.2 ゲームを作ろう

「プログラミング及び実習 I」も残すところ 3 回となりました。今回は、これまでの授業で勉強したことを応用して、次のようなゲームを作ってみることにします。

数当てゲーム このゲームは、コンピュータ (このゲームプログラム) が勝手に選んだ整数を、ユーザーが推測して当てるといったものです。

1. まず、コンピュータは 1～9 の 9 種類の数字から勝手に 3 種類の数字を選び、この 3 つを勝手に並べて 3 桁の整数を作ります。つまり、この 3 桁の整数には 0 という数字は使われていませんし、同じ数字が 2 度以上使われていることもありません。コンピュータの選んだ 3 桁の整数が問題の正解になります。



1回目 ? 123

1H0B

2回目 ? 456

0H0B

3回目 ? 789

0H2B

4回目 ? 178

0H1B

5回目 ? 923

1H1B

6回目 ? 827

1H0B

7回目 ? 893

おめでとう。正解です。

2. ユーザーは、コンピュータの選んだ 3 桁の整数を推測し、それを入力します。最初は全く手がかりがありませんので適当に整数を選んで入力しなければなりません。入力として許されるのは、コンピュータが選ぶことのできる形の 3 桁の整数でなければなりません。つまり、0 を含んでいたり、同じ数字を 2 度以上使っているような整数を入力してはいけません。
3. ユーザーが 3 桁の整数を入力すると、コンピュータは次のような 2 つのヒントを返します。

Hit 数 数字が正解と一致しているような桁の個数

Blow 数 正解では別の桁にその数字が使われているような桁の個数

たとえば、正解が 123 で、ユーザーの推測 (入力) が 132 の場合は、Hit 数は 1、Blow 数は 2 となります。

4. 正解が入力される (Hit 数が 3 になる) まで、2 と 3 が繰り返されます。ユーザーは 10 回以内に正解を入力しなければなりません。

右は、このゲームプログラム (hitblow.c) の実行例です。コンピュータが選んだ数 (正解) は 893 でした。

「... 回目 ?」というプロンプト (入力を促すメッセージ) の後の 3 桁の整数がユーザーの推測 (入力) です。入力された整数の Hit 数が n 、Blow 数が m の場合、プログラムは $nHmB$ のようにヒントを表示しています。7 回目の入力 893 は正解でしたので、ここでプログラムが終了しています。

メモ

数当てゲームのプログラム では、このゲームプログラムのソースファイル hitblow.c を紹介します。ただし、行頭の整数は説明のために付けた行番号で、このプログラムの一部ではありません。また、ソースプログラム中で `/*` と `*/` とで囲まれた部分は注釈 (プログラムの説明) です。C 言語では、このようにしてプログラム中に任意の注釈を書きしておくことができます。この部分はプログラムとしては無視されてコンパイルされます。

hitblow.c

```
1 #include <turtle.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 int main()
6 {
7     int try, hit, blow;
8     int x1, x2, x3, g1, g2, g3;
9     int guess;
10
11     /* 現在時刻を乱数の種として設定 */
12     srand(time(0));
13
14     /* 正解(1~9の数字3個の重複のない並び)を乱数で生成する */
15     x1 = rand()%9+1;          /* 左端 */
16     do {
17         x2 = rand()%9+1;      /* 真中 */
18     } while (x2 == x1);      /* 同じ数だったらもう一度 */
19     do {
20         x3 = rand()%9+1;      /* 右端 */
21     } while (x3 == x1 || x3 == x2); /* 同じ数だったらもう一度 */
22
23     tPenUp();
24     tMoveTo(-100, 200);
25
26     for (try = 1; try <= 10; try++) {
27         /* キーボードから3桁の数を入力 */
28         tPrintf("%d回目 ? ", try);
29         tScanf("%d", &guess);
30
31         /* 各桁を取り出す */
32         g1 = guess/100;      /* 左端 */
33         g2 = guess/10%10;    /* 真中 */
```

```

34      g3 = guess%10;          /* 右端 */
35
36      /* 1~9の数字3個の並びかどうかをチェックする */
37      if (g1 > 9 || g1 < 1 || g2 < 1 || g3 < 1)
38          continue;
39
40      /* その3個の数字に重複がないかどうかをチェックする */
41      if (g1 == g2 || g2 == g3 || g3 == g1)
42          continue;
43
44      /* hit と blow の数を数える */
45      hit = blow = 0;
46      if (g1 == x1) hit++;
47      if (g2 == x2) hit++;
48      if (g3 == x3) hit++;
49      if (g1 == x2 || g1 == x3) blow++;
50      if (g2 == x3 || g2 == x1) blow++;
51      if (g3 == x1 || g3 == x2) blow++;
52
53      /* 正解なら繰り返しを終了 */
54      if (hit == 3)
55          break;
56
57      /* hit と blow の数を表示 */
58      tPrintf("%dH%dB\n", hit, blow);
59  }
60
61      /* 10回以内で正解が入力されなかったら、正解を表示する。 */
62      if (try <= 10)
63          tPrintf("おめでとう。正解です。 \n");
64      else
65          tPrintf("残念でした。正解は %d%d%d です。 \n", x1, x2, x3);
66  }

```

メモ

乱数の発生 まず、正解となる3桁の整数を作らなければなりません。このプログラムでは、その整数の各桁を左から順に x1、x2、x3 という3つの変数に記憶しています。各変数には1から9までの数字を適当に選んで記憶しておかなければなりません。そこで必要となるのが乱数と呼ばれるでたらめな数です。C というプログラミング言語では rand() という式を使うことで、このでたらめな(0以上の)整数を生成することができます。たとえば、

```
x1 = rand();
```

とすれば、変数 x1 にでたらめな0以上の整数が格納されます。このゲームに必要なのは、1から9までの数(字)ですから、

```
x1 = rand()%9+1;
```

とすることでこれを実現できます。でたらめな整数を9で割ったあまりが0から8までの(でたらめな)数となり、これに1を足すことで1から9までの(でたらめな)数となるからです。これを行っているのが、`hitblow.c` の15行目です。

`rand()` という式の値は、それが計算される度にでたらめな整数となりますので、真中の桁の数字 `x2` も同じように `rand()` を使って作ることができます。ただし、このゲームでは3桁の中に同じ数字を2回以上使うことは許されませんから、`x2` が `x1` と同じになったら、もう一度

```
x2 = rand()%9+1;
```

を実行して、別の数字が生成されるまでこれを繰り返します。この繰り返しが16～18行目の `do` 文です。右端の桁 `x3` の場合は、同様に `x1` と `x2` と異なる数字が生成されるまで繰り返すこととなります(19～21行目)。

メモ

乱数の種 `rand()` という式が生成する整数は、見かけは乱数のように見えますが、実はある規則にしたがって順に決った整数が生成されているに過ぎません。つまり、プログラムの実行が開始されてから `rand()` が1番目、2番目、3番目、... と生成していく乱数は常に同じ数列になります。これでは何度このゲームプログラムを実行しても、正解の3桁の整数は常に同じものになってしまいますので、`hitblow.c` の12行目では、現在時刻¹を表す `time(0)` という式の値を乱数の種として設定するという処理を行っています。`rand()` が生成する乱数列は、`srand` によって設定された整数(乱数の種)によって変わりますので、プログラムが実行されたときの時刻に応じて正解も変わってくるようになります。

プログラムの中で、乱数を発生する `rand()` や、乱数の種を設定する `srand()` を使う場合は、プログラムの冒頭部分に「`#include <stdlib.h>`」を、現在時刻を取得する `time()` を使う場合は「`#include <time.h>`」を書いておく決まりになっています(2～3行目)。

メモ

continue 文 `hitblow.c` の36行目から42行目では、キーボードから入力された数が、1から9までの数字3個の重複のない並びであるかどうかをチェックしています。このチェックを通らなかった場合に実行されている `continue;` という文は、そのとき `while` や `for`、`do` によって繰り返されている文(ブロック)の残りの部分を飛び越して、その回の実行をそこで終えるための構文です。

¹世界標準時の1970年1月1日00:00:00からの経過秒数で現在時刻を表します。

for 文の中で `continue;` が実行された場合、「再設定式」(`hitblow.c` の場合は 26 行目の `try++`) が実行されてから、「継続条件式」が調べられます。もし、「継続条件式」が成り立っていれば、繰り返しの次の回が開始されます。成り立っていなければ、for 文の実行は終了します。また、while 文や do 文の中で `continue` 文が実行された場合は、すぐに、その while 文や do 文の条件式が調べられ、成り立っていれば、繰り返しの次の回が開始されます。成り立っていなければ、while 文や do 文の実行は終了します。

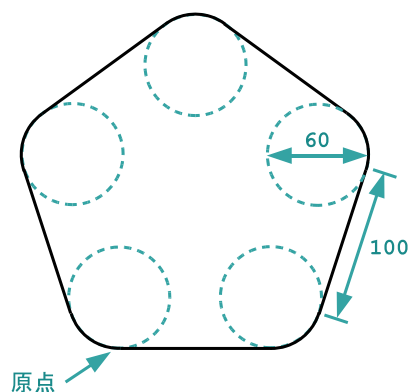
メモ

6.3 演習問題 — 注意：AI にプログラムを作成してもらうのでは意味がありません

1. ウィンドウ内を連続して n 回クリックすると、下図のような角の丸い n 角形を描くプログラム `rpolygon.c` を作成し、コンパイル、実行して、正しく動作することを確認しなさい。ただし、while 文ではなく、for 文を使ったプログラムにしなさい。また、`i = i+1` の代りに `i++` を使うなど、今回解説したものが(効果的に)使えるところがあれば、できるだけ使うようにしなさい。

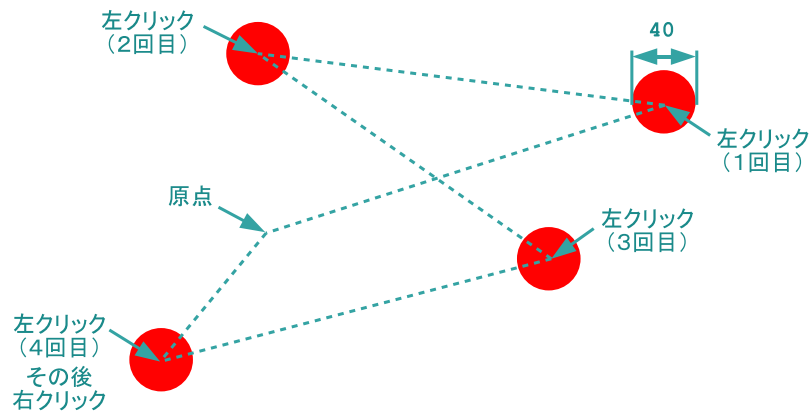
この図形は、長さ 100 の線分と、半径 30 の円周の一部(円弧)をつなぎ合わせたものとなっています。原点の位置は、このつなぎ目の 1 つになっています。

5回連続してクリックした場合



円周の一部(円弧)は `tArc` という関数(この科目のタートルグラフィックスの機能の 1 つ)を呼び出すことで描くことができます。この関数の使い方については、第 5 回配布資料の付録「タートルグラフィックスの機能一覧」を参照してください。

2. ウィンドウ内を左クリックする度に、クリックされた位置を中心とする半径 20 の赤い円盤を描いていくプログラム `dots.c` を作成し、コンパイル、実行して、正しく動作することを確認しなさい。ただし、左ボタン以外でクリックされると、カメは仕事を止めて(ペンを上げて)原点に戻るようしなさい。



ヒント：円盤 (内部が塗り潰された円) は `tMark`、`tCircle`、`tFill` の3つの関数の呼び出しを組み合わせることで描くことができます。また、塗り潰しの色を赤色に変更するためには、`tSetColor` を呼び出すようにしてください。たとえば

```
#include <turtle.h>

int main()
{
    tSetColor(1.0, 0.0, 0.0);
    tPenUp();
    tMoveTo(100, 100);
    tForward(50);
    tTurn(90);
    tMark();
    tCircle(50);
    tFill();
}
```

というプログラムは、(100, 100) を中心とする半径 50 の赤い円盤を描きます。ペンの上下は、関数 `tFill` による塗り潰しには影響しません。`tMark` を呼び出しているのは、続く `tCircle` の呼び出しで1周する円周の内部だけを塗り潰したいからです。`tMark` を呼び出しておかないと、原点からのカメの移動の軌跡すべての内部が塗り潰されてしまいます。これらの関数の詳細については、第5回配布資料の付録の「タートルグラフィックスの機能一覧」を参照してください。

6.4 今回の実習内容

1. プリントをもう一度読み返しましょう。例題 `hitblow.c` のソースプログラムを作成し、コンパイル、実行してみましょう。プログラムが完成したら「課題の提出と確認」の Web ページから提出してください。
2. 演習問題に取り組みましょう。プログラムが完成したら、「課題の提出と確認」の Web ページからの提出を忘れないでください。
3. クイズに答えてください。前回までと同様に「課題の提出と確認」の Web ページで「第6回クイズ」を選択し、「送信」のボタンをクリックしてクイズに答えてください。
4. 以下の要領で、自由に魅力的なプログラムを作成してください。

- ソースファイルの名前は `final.c` としてください。提出時に自分のプログラムにタイトルを付けていただきます。
- もちろん、マウスやキーボードからの入力を使っても ok です。ただし、どのように操作すればよいか、そのプログラムを実行しただけで分かるように工夫してください。
- ソースファイルには他人や AI が書いたプログラムが含まれてはいけません。
- この科目で紹介したことのない関数は(基本的には)使用できません。もし、どうしても使用したい場合は、次の授業終了までに担当教員に相談してください。
- プログラムが完成したら、7月27日(月) 23:59 までに「課題の提出と確認」の Web ページから提出してください。
- みなさんが作ったプログラム(ソースプログラムとオブジェクトプログラム)は、7月30日(木)から匿名で、情報実習室やセルフラーニング室の Windows PC で公開します。
- 公開期間中、1人10件程度、他の受講者が作成したプログラムを評価していただきます。この評価も課題の一部です。3号館地下のセルフラーニング室等を利用して、プログラムを実行してみて、「課題の提出と確認」の Web ページから、8月4日(火) 16:30 までに忘れず評価を行ってください。
- 評価は、次のような観点で行い、自分が評価を担当していないプログラムも含めて、ごく平均的と思われるプログラムの点数が、満点の半分になるように採点してください。
 - (a) プログラムを実行した時の動作や、その描く図形などは魅力的か(10点満点)
 - (b) ソースプログラムにこの科目で勉強したことが反映されているか(5点満点)

次のフォルダに過年度の作品がいくつか置いてありますので参考にしてください。

R:\a89023\過年度自由課題

みなさんが作成した作品も、来年度以降の受講者に対して同様に紹介させていただきます。

```
#include <turtle.h>

int main()
{
    int bet;
    int x, y;

    tPenUp();
    tMoveTo(100, 100);
    tPenDown();
    tPrintf("Come here");
    tMoveTo(100, 120);
    tMoveTo(200, 120);
    tMoveTo(200, 80);
    tMoveTo(100, 80);
    tMoveTo(100, 100);
    tPenUp();
    tMoveTo(0, 0);
    tPenDown();
    flag = 0;
    while (tGetClick() == 1) {
        x = tClickX();
        y = tClickY();
        tMoveTo(x, y);
        if (100 < x && x < 200 && 80 < y && y < 120)
            bet = 1;
    }
    if (bet)
        tPrintf("You bet\n");
    else
        tPrintf("No way\n");
}
```

```
1 #include <turtle.h>
2
3 int main()
4 {
5     int m, n, k;
6
7     tPenUp();
8     tBackward(250);
9     tPrintf("いくつまで? ");
10    tScanf("%d", &n);
11    tPrintf("%d以下の素数は", n);
12    m = 2;
13    while (m <= n) {
14        k = 2;
15        while (k < m) {
16            if (m % k == 0)
17                break;
18            k = k + 1;
19        }
20        if (k == m)
21            tPrintf(" %d", m);
22        m = m + 1;
23    }
```

```
24     if (n > 1)
25         tPrintf("  です\n");
26     else
27         tPrintf("ありません\n");
28 }
```

このプログラムは、2以上の整数 n の入力を終えると、まず「... 以下の素数は」という文字列を表示し、その後、2から n までの整数 m について、順に素数であるかどうかを調べ、素数である場合のみ、その m を表示するようにしています。変数の名前は違いますが、このプログラムの14～19行目は、前回の例題 `smartprime.c` からの引き写しであることに注意しましょう。15～19行目の `while` 文を抜け出たときに、変数 k の値が m と等しいということは、2から $m-1$ までの整数では m が割りきれなかった (`break` 文が実行されなかった) ということですから、その時に限って m の値を表示しています。